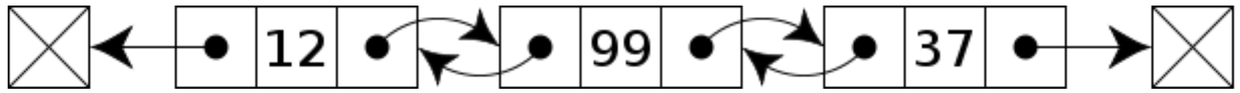
Google Reichman University | AI Tech School

Double Link List

Izik Zur

Definitions and Terms

- Linked List:
- container, insert / remove by key, grows on demand
- Definition
  - Node: Data + Pointers (forward, backward)
  - Head, Tail



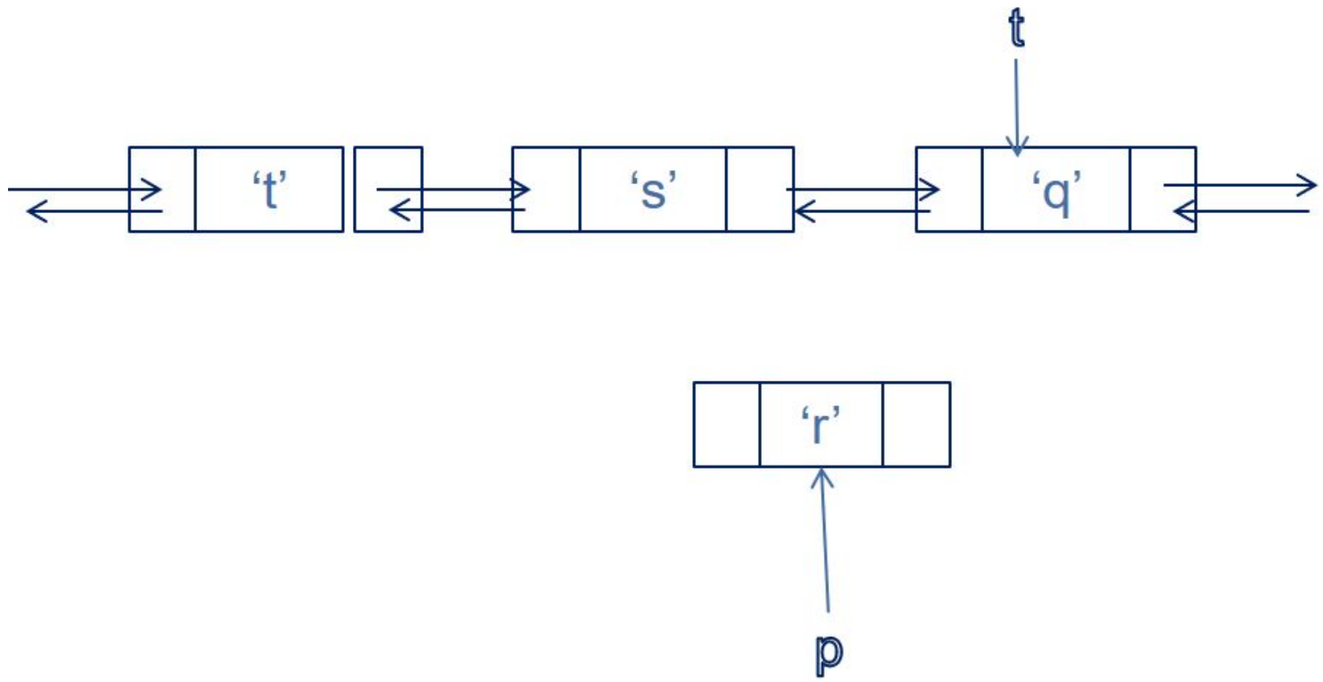
Pros & Cons

- **Pros**
  - Dynamic Memory management (consumes as required)
  - Efficient in Sorted list (add/remove)

- **Cons**
  - More Space, no direct access

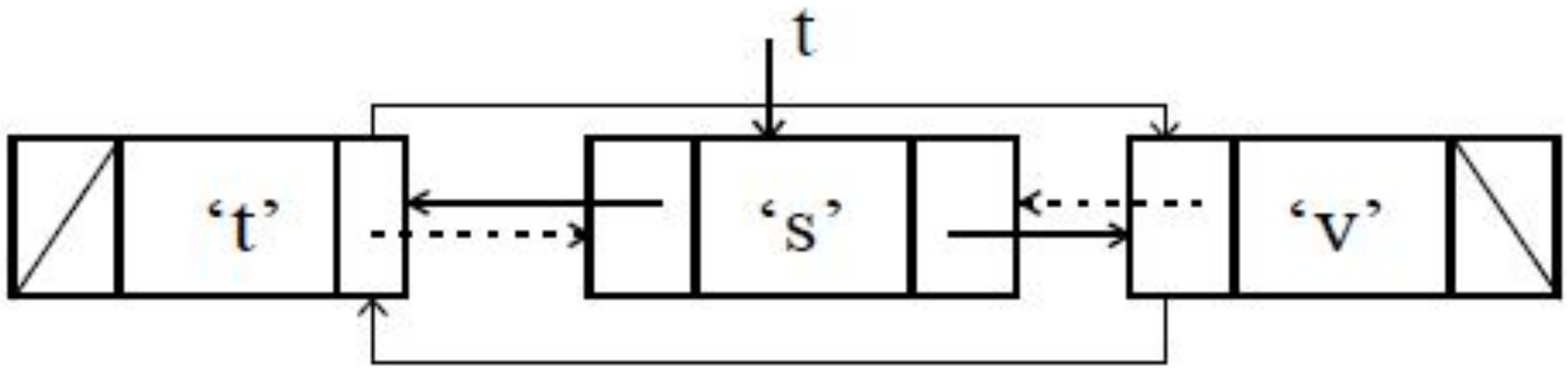
Insert

```
p-> prev = t->prev;
p-> next = t ;
t-> prev->next = p;
t->prev = p;
```



Remove

```
t → next →prev = t → prev;
t → prev → next = t → next;
```



Node Structure

```
struct Node
{
void*       m_data;
Node*       m_next;
Node*       m_prev;
};
```

List Structure

```
struct List
{
Node m_head;
Node m_tail;
};
```

API's (1)

```
typedef struct List List ;
typedef void* ListItr;
typedef int (*ListActionFunction)(void* _element, void* _context);


List* ListCreate(void);
void ListDestroy(List** _pList, void (*_elementDestroy)(void* _item));
ListItr ListPushHead(List* _list, void* _item);
ListItr ListPushTail(List* _list, void* _item);
void* ListPopHead(List* _list);
void* ListPopTail(List* _list);
```

API's (2)

```
ListItr ListItrBegin(const List* _list);
ListItr ListItrEnd(const List* _list);

ListItr ListItrNext(ListItr _itr);
ListItr ListItrPrev(ListItr _itr);

void* ListItrGet(ListItr _itr);
void* ListItrSet(ListItr _itr, void* _element);

ListItr ListItrInsertBefore(ListItr _itr, void* _element);
void* ListItrRemove(ListItr _itr);
```

API’s (3)

```
size_t ListSize(const List* _list);
IntListIsEmpty(const List* _list);

ListItr ListItrForEach(ListItr _begin, ListItr _end, ListActionFunction
_action, void* _context);
```

Google Reichman University | AI Tech School

Exercise

Implement Double Linked List

THANK YOU

GOOD LUCK